TASKPILOT

Application de gestion de tâches Full-Stack

Boisson Yann
Formation DWWM
2026

Contexte du projet

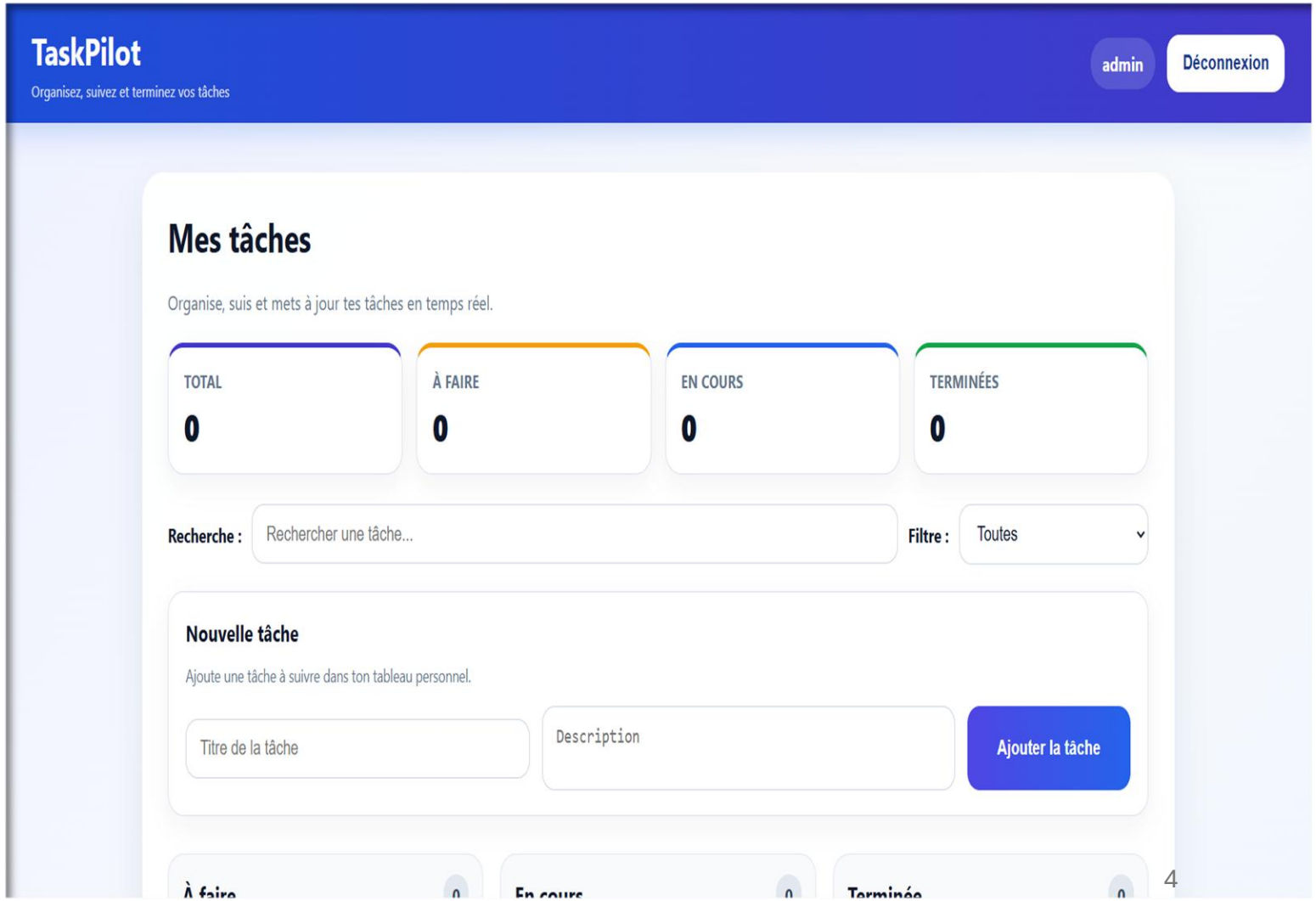
- Projet réalisé en formation DWWM
- Application web full-stack
- Gestion des tâches personnelles
- Interface interactive Kanban

Objectifs

- Permettre la gestion de tâches simplement
- Créer une interface moderne et intuitive
- Implémenter un système Drag & Drop interactif
- Sauvegarder les tâches en base de données
- Sécuriser l’accès avec authentification

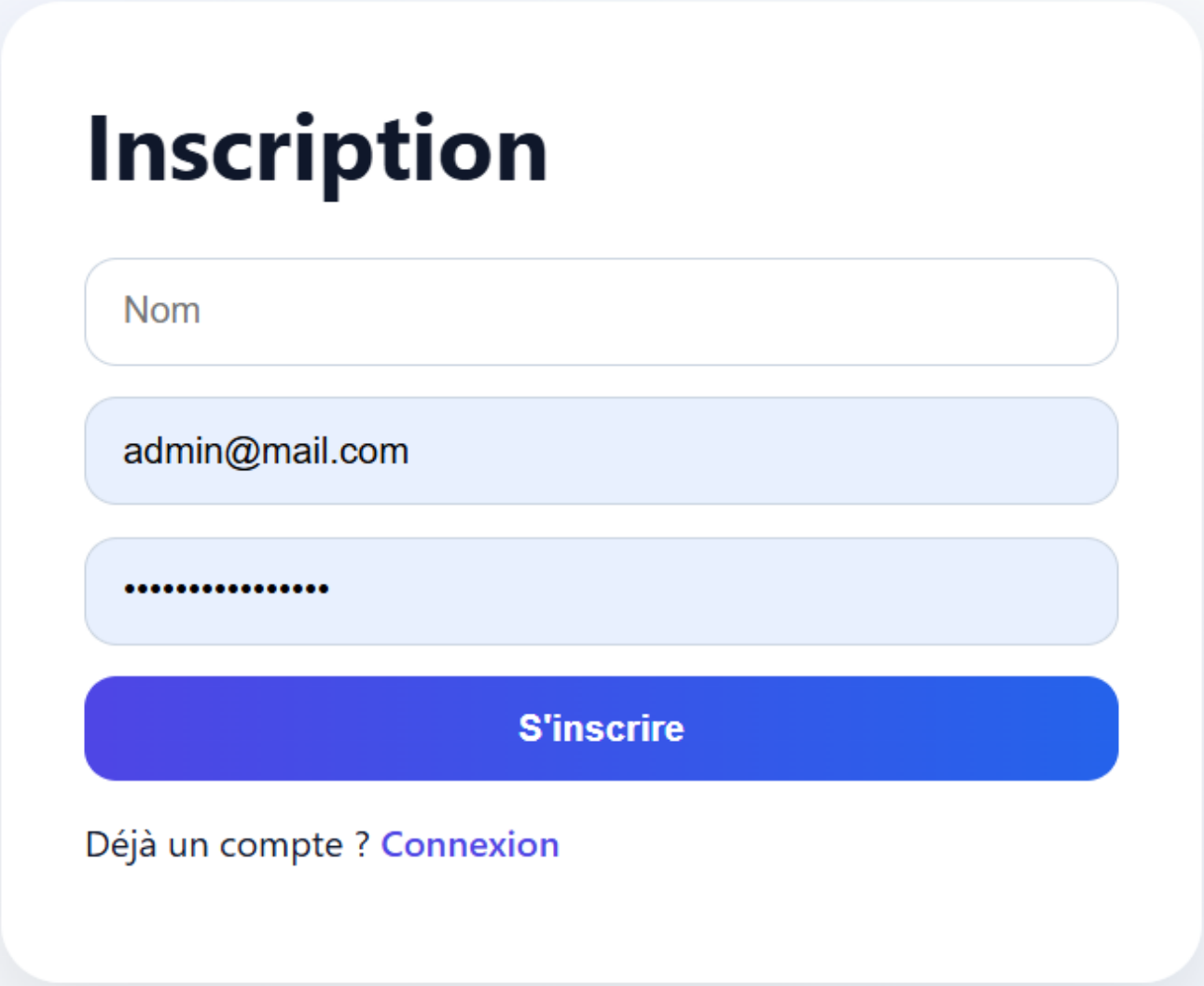
Fonctionnalités principales

- Ajouter une tâche
- Modifier le statut
- Supprimer une tâche
- Recherche et filtres
- Statistiques
- Déplacer les tâches en Drag & Drop



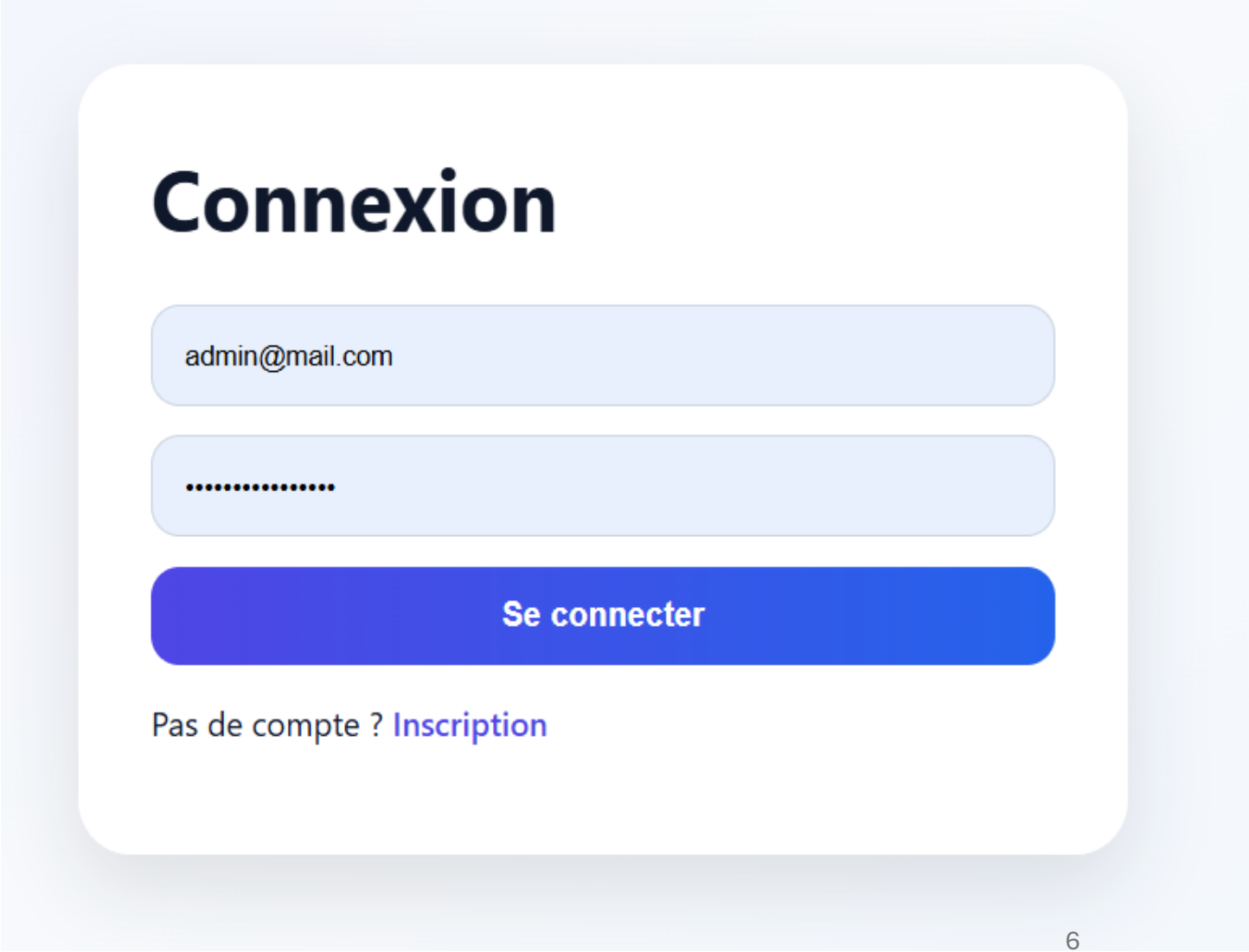
Inscription

- Création de compte utilisateur
- Stockage sécurisé du mot de passe (hash)
- Lien vers connexion



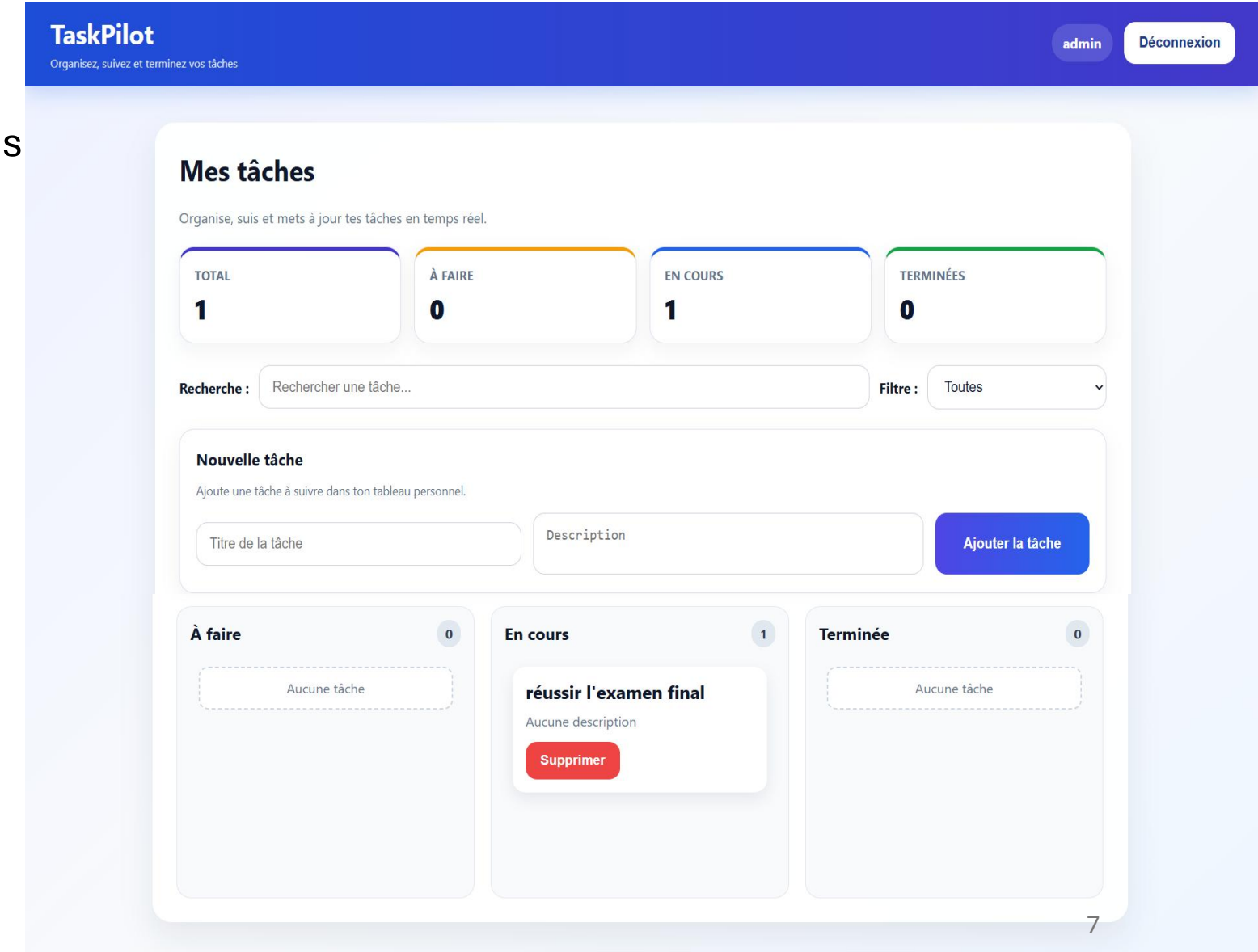
Connexion

- Interface de connexion utilisateur
- Authentification via email et mot de passe
- Validation des informations côté backend



Dashboard

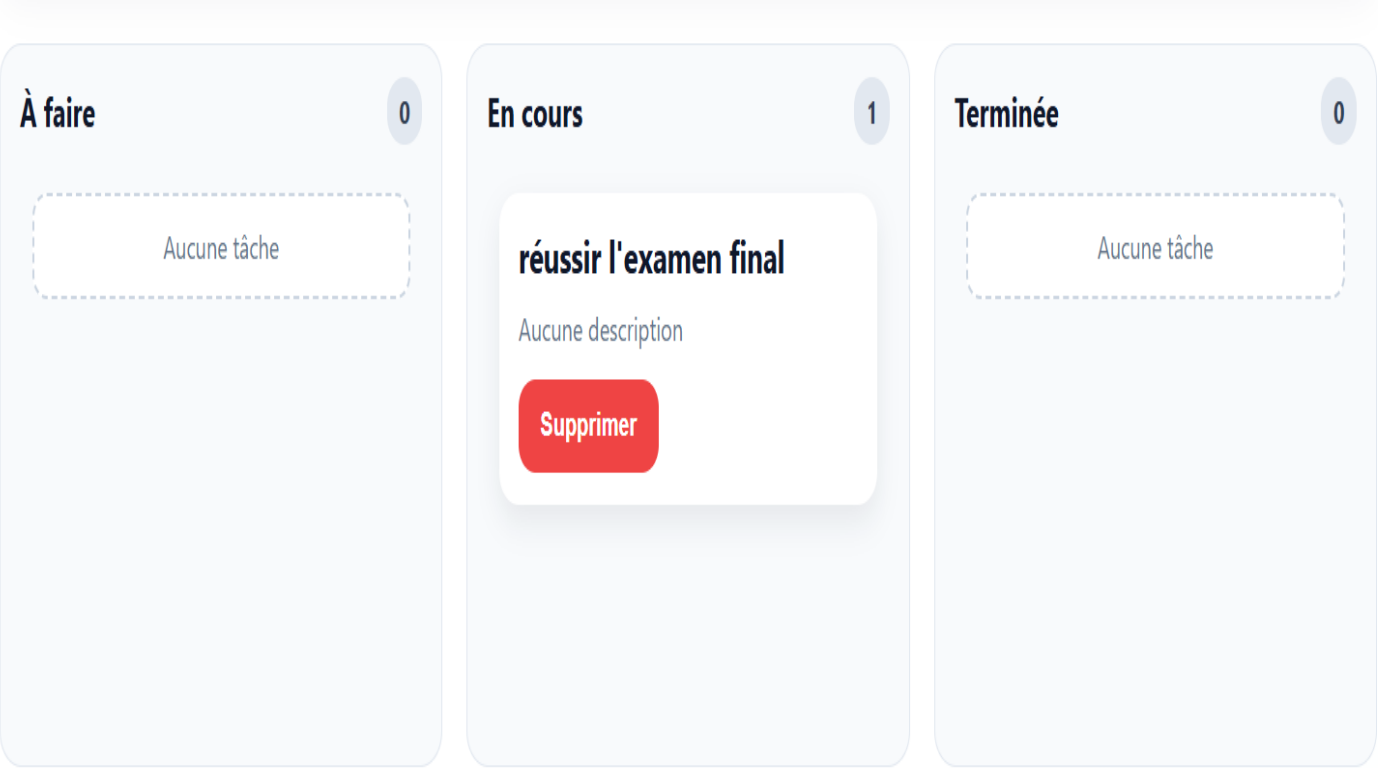
- Tableau principal de gestion des tâches
- Affichage des statistiques
- Interface claire et responsive



Kanban

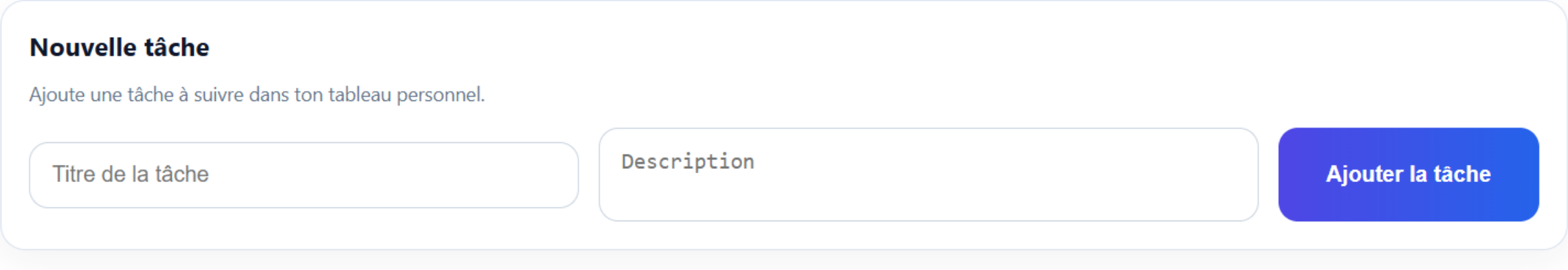
Organisation des tâches en colonnes :

- **À faire**
- **En cours**
- **Terminées**
- Déplacement des tâches par **drag & drop**



Ajout de tâches

- Formulaire **d'ajout de tâche**
- **Titre + description**
- Ajout en temps réel



Maquette LOGIN desktop

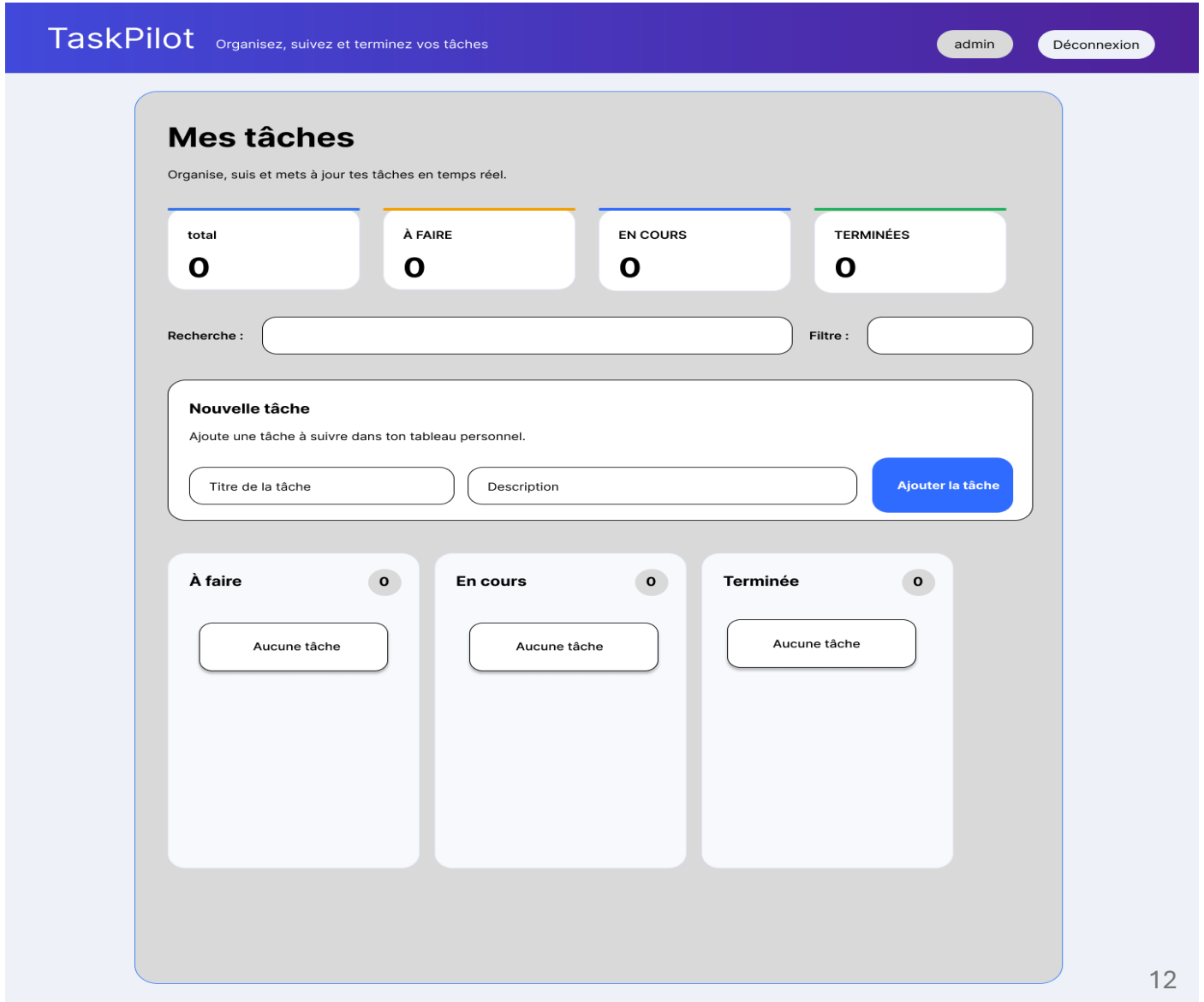
- Maquette réalisée sur **Figma**
- **Design centré**
- **Formulaire simple et lisible**

Maquette LOGIN mobile

- Adaptation **responsive**
- **Optimisation** pour smartphone
- Lisibilité améliorée

Maquette DASHBOARD desktop

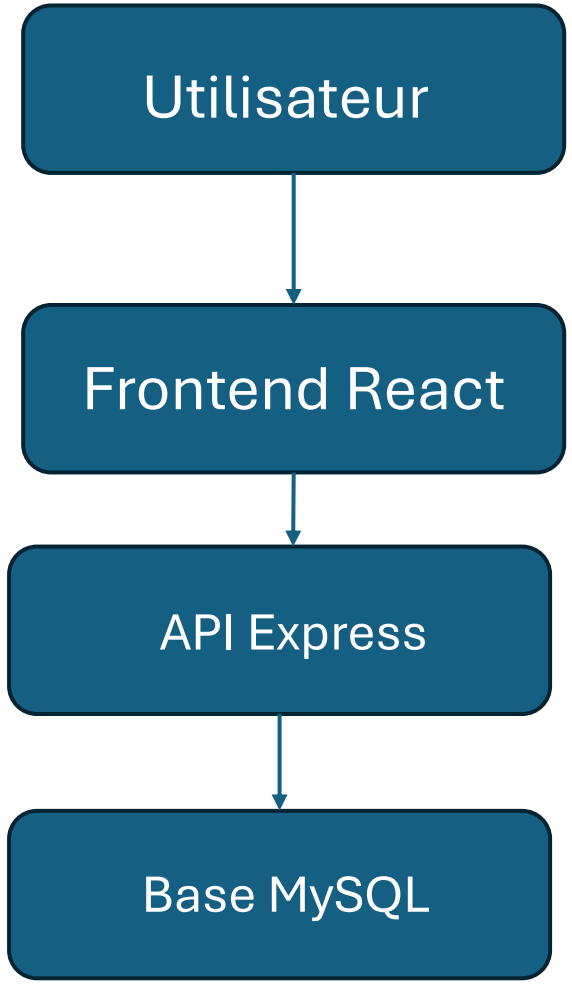
- Disposition en grille
- Statistiques visibles
- Interface ergonomique



Maquette DASHBOARD mobile

- **Disposition verticale**
- **Cartes empilées**
- Navigation simplifiée

Architecture



Explication architecture

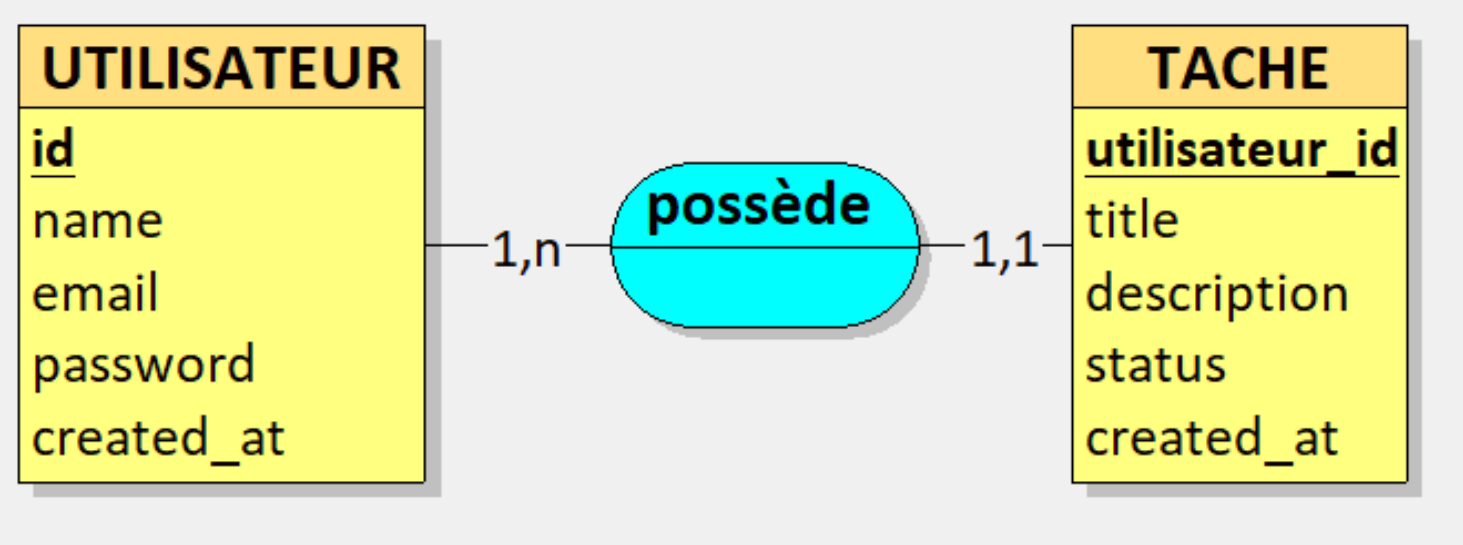
- **React** gère l’interface utilisateur
- **Express** gère les requêtes **API**
- **MySQL** stocke les données
- **Communication via HTTP**

Modèle Conceptuel des Données (MCD)

Entités principales :

- **Utilisateur**

- **Tâche**

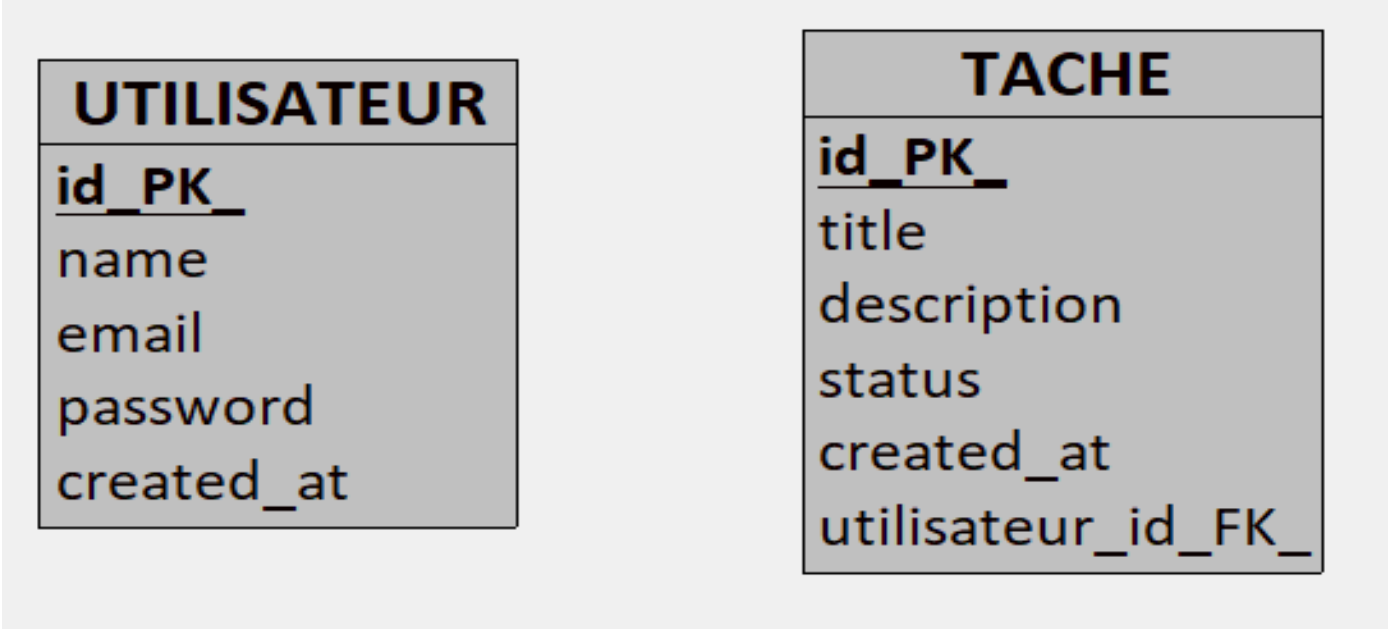


- Relation : **un utilisateur possède plusieurs tâches**

Modèle Logique des Données (MLD)

- **UTILISATEUR** (id, name, email, password)

- **TACHE** (id, title, description, status, utilisateur_id)

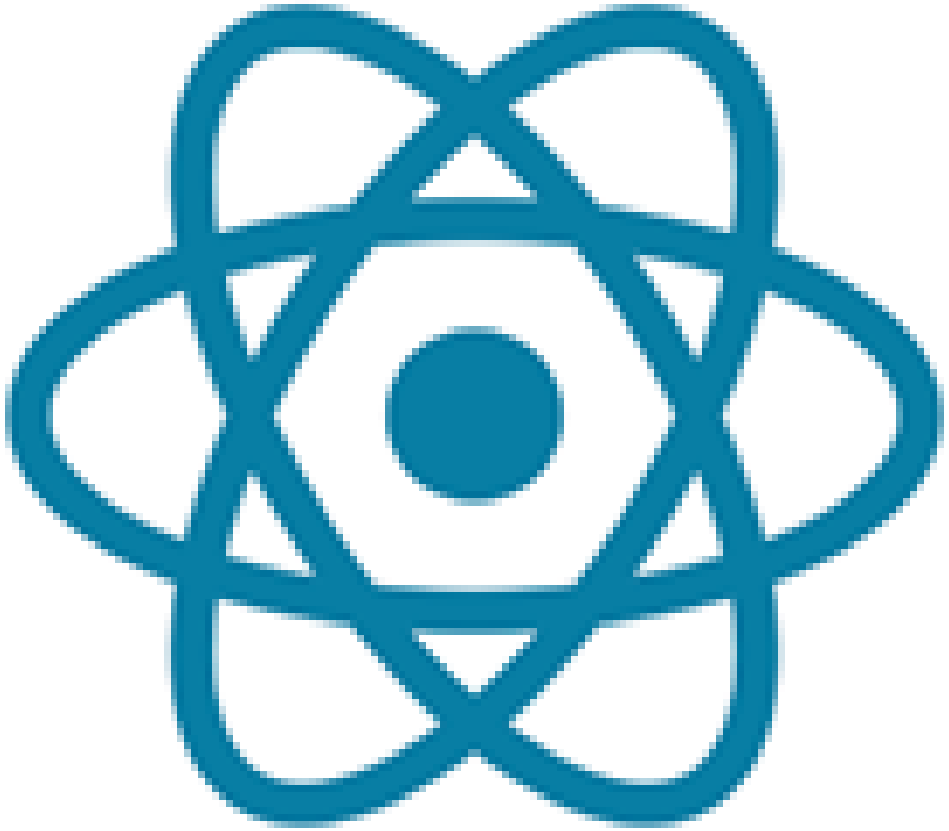


Relation

- 1 utilisateur → plusieurs tâches
- Clé étrangère : utilisateur_id
- Relation 1,N

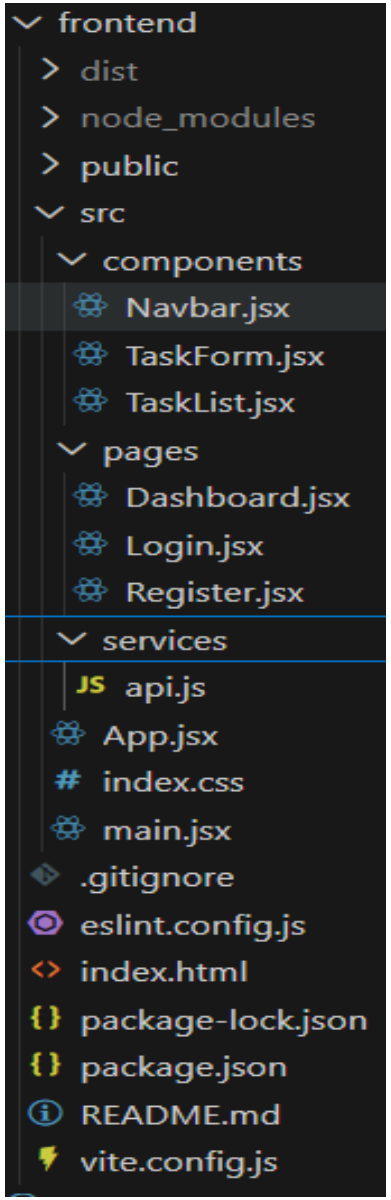
Frontend

- Développé avec **React**
- Composants réutilisables
- Gestion dynamique de l’interface



Structure

- **Composants**
- **Pages**
- **Services API**
- **Gestion des états**



Code Formulaire

- Formulaire **React** contrôlé
- Utilisation de **useState**
- **Gestion des inputs utilisateur**

```
{/* Formulaire d'ajout de tâche */}
<form onSubmit={handleSubmit} className="task-form">
    <input
        type="text"
        placeholder="Titre de la tâche"
        value={title}
        onChange={(e) => setTitle(e.target.value)}
    />

    <textarea
        placeholder="Description"
        value={description}
        onChange={(e) => setDescription(e.target.value)}
    />

    <button className="primary-btn" type="submit">
        Ajouter la tâche
    </button>
</form>
```

Gestion State

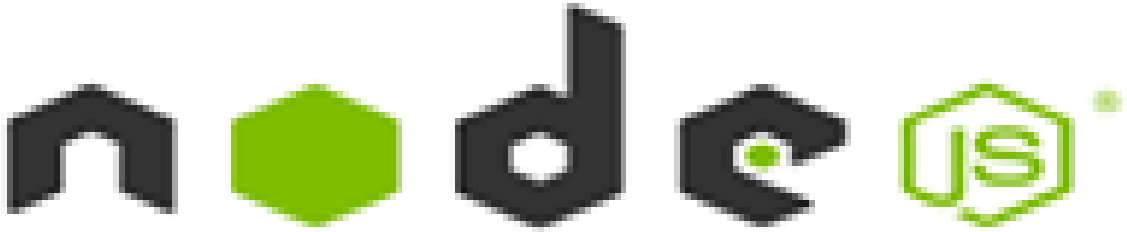
- **useState** pour gérer les données
- Mise à jour en temps réel
- Synchronisation avec backend

```
// État du champ titre
const [title, setTitle] = useState("");

// État du champ description
const [description, setDescription] = useState("");
```

Backend

- Développé avec **Node.js** & **Express**
- **API REST**
- Gestion des **routes**



Routes API

- **POST** /tasks → ajouter
- **GET** /tasks → récupérer
- **DELETE** /tasks → supprimer

```
// Crée une nouvelle tâche
router.post("/", (req, res) => {
    const { title, description, status } = req.body;

    if (!title) {
        return res.status(400).json({ message: "Le titre est obligatoire" });
    }
```

```
// Supprime une tâche
router.delete("/:id", (req, res) => {
    const taskId = req.params.id;

    db.query(
```

```
// Récupère toutes les tâches de l'utilisateur connecté
router.get("/", (req, res) => {
    db.query(
        "SELECT * FROM tasks WHERE user_id = ? ORDER BY created_at DESC",
        [req.user.id],
        (err, results) => {
            if (err) {
                return res.status(500).json({ message: "Erreur serveur" });
            }
            res.json(results);
        }
    );
```

MYSQL

- **Connexion base MySQL**
- **Stockage des utilisateurs**
- **Stockage des tâches**

```
// Connexion à la base de données MySQL
const db = mysql.createConnection({
    host: process.env.DB_HOST,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_NAME,
});
```

AUTH MIDDLEWARE

- Vérification du **token JWT**
- Protection des **routes**
- **Accès sécurisé**

```
// Middleware qui protège les routes nécessitant une authentification
const authMiddleware = (req, res, next) => {
    const authHeader = req.headers.authorization;

    // Vérifie que le header Authorization existe
    if (!authHeader || !authHeader.startsWith("Bearer ")) {
        return res.status(401).json({ message: "Accès refusé" });
    }
```

JWT

- **Token** généré à la connexion
- Envoyé dans les **requêtes**
- Permet d'identifier **l'utilisateur**

```
const token = authHeader.split(" ")[1];

try {
    // Vérifie et décode le token JWT
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    // Ajoute les informations utilisateur dans la requête
    req.user = decoded;
```

Sécurité

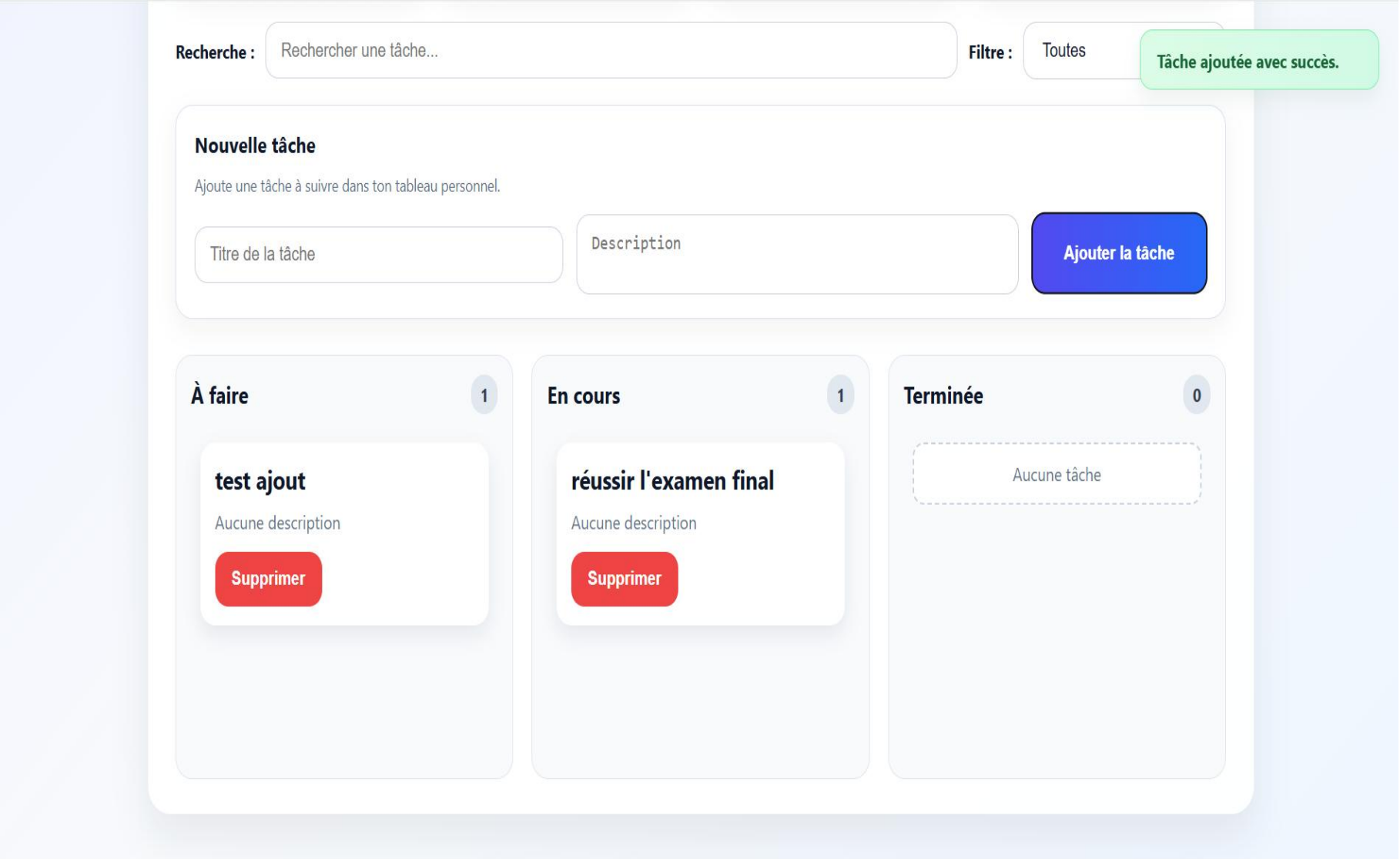
- Authentification avec **JSON Web Token**
- Mot de passe **hashé**
- **Middleware** de **protection des routes**
- Chaque utilisateur voit uniquement ses tâches

Test de connexion

- **Connexion réussie**
- **Accès au dashboard**
- Gestion des erreurs

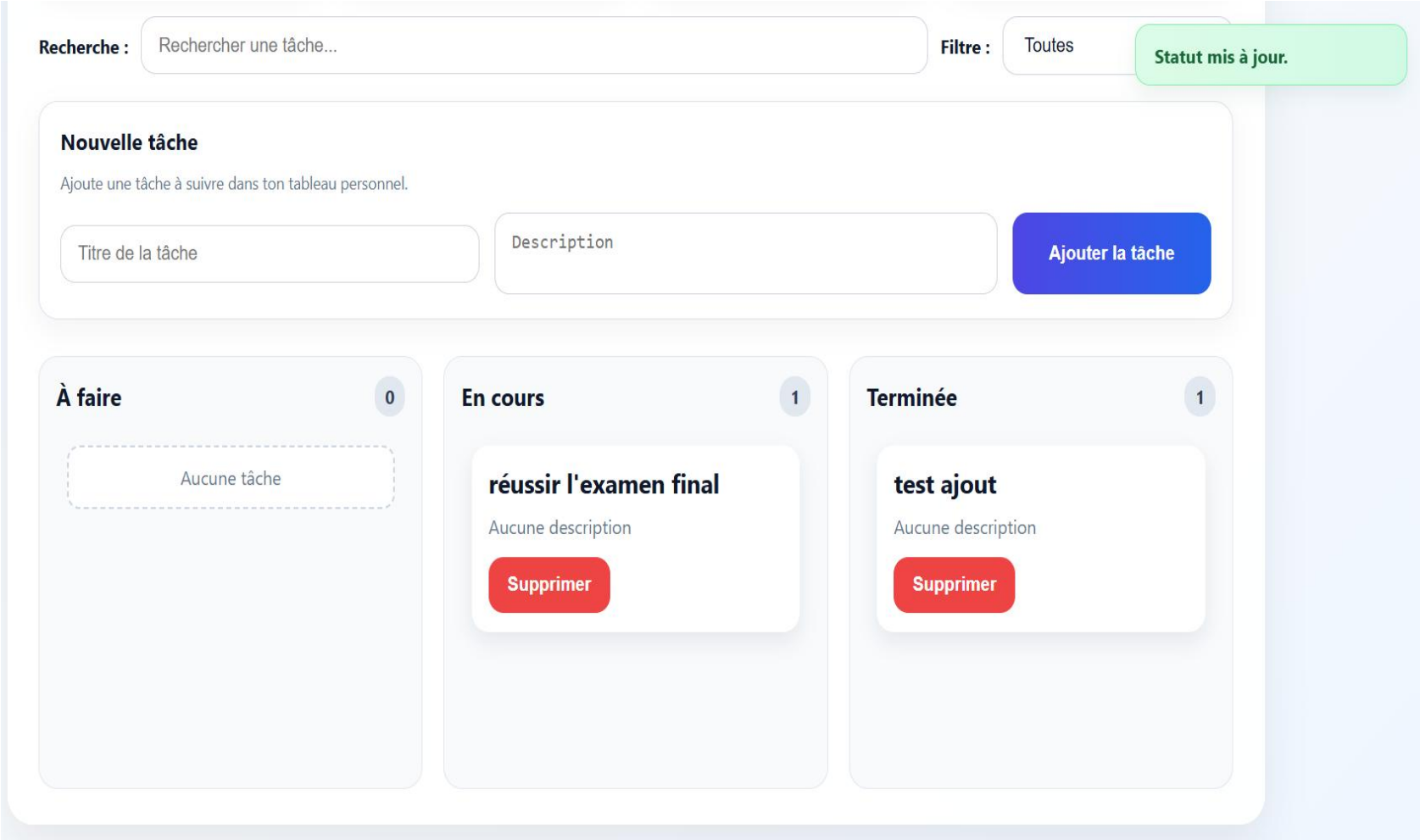
Test ajout

- **Ajout d’une tâche**
- **Affichage immédiat**
- **Stockage en base**



Test Drag & Drop

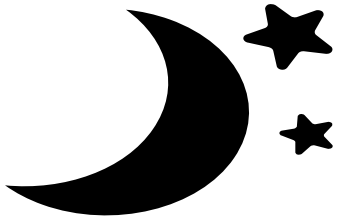
- **Déplacement entre colonnes**
- **Mise à jour du statut**
- **Expérience utilisateur fluide**



Technologies utilisées

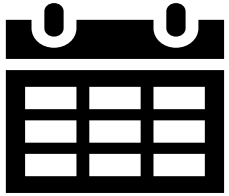
- **React** (interface utilisateur)
- **DnD-Kit** (Drag & Drop)
- **Node.js** & **Express** (API backend)
- **MySQL** (base de données)
- **JWT** (authentification sécurisée)
- **CSS** personnalisé (design responsive)

Améliorations possibles

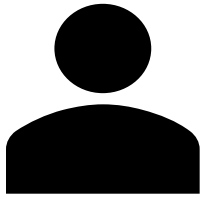


- Mode sombre

- Version mobile (native)



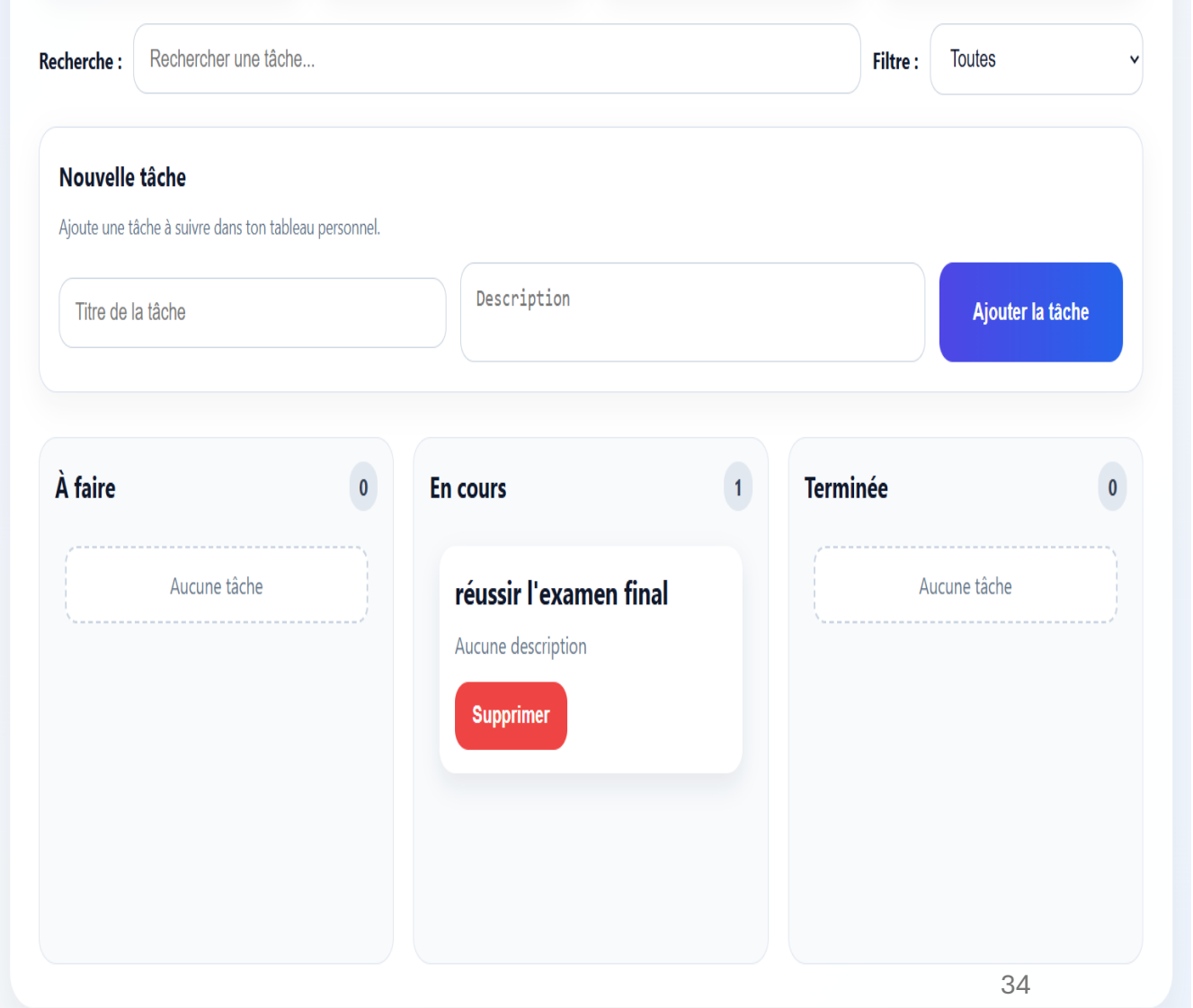
- Ajout de dates limites



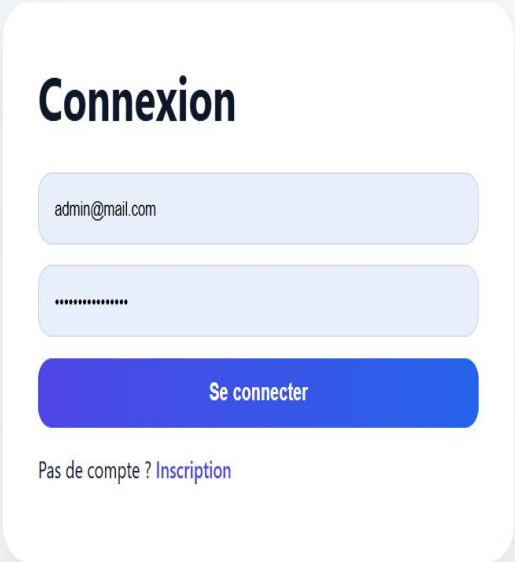
- Notifications utilisateur
- Gestion multi-utilisateurs avancée

Conclusion

- Réalisation d’une application Full-Stack complète
- Mise en pratique des compétences frontend et backend
- Application fonctionnelle et interactive
- Projet enrichissant et formateur



Démonstration

Présentation rapide de l’application :

- Connexion utilisateur
- Ajout d’une tâche
- Drag & Drop
- Suppression
- Statistiques